

Capítulo 5: Estimação de Parâmetros

Bioinformática para Biologia de Sistemas

Ney Lemke

DBF-Unesp

8 de junho de 2026

Sumário

- 1 Introdução
- 2 Formulação do Problema
- 3 Busca em Grade (Grid Search)
- 4 Métodos de Gradiente
- 5 Algoritmos Globais
- 6 Análise de Identificabilidade
- 7 Validação e Critérios de Ajuste
- 8 Exercícios
- 9 Referências

5.1 Por que Estimação de Parâmetros?

Objetivos de Aprendizagem

- Compreender o problema inverso em biologia de sistemas
- Dominar métodos de otimização local e global
- Aplicar técnicas de estimação a modelos biológicos
- Avaliar identificabilidade e incerteza de parâmetros
- Validar modelos com critérios estatísticos

Definição: Estimação de Parâmetros

Processo de determinar valores numéricos de parâmetros de um modelo matemático a partir de dados experimentais, ajustando o modelo para reproduzir o comportamento observado do sistema.

O Problema Inverso

Problema Direto:

- Parâmetros conhecidos
- Condições iniciais dadas
- Simular $\rightarrow$ Previsões
- Determinístico

Exemplo

Dados $k_1 = 0.5$, $k_2 = 0.1$

Simular: $\frac{dx}{dt} = k_1 - k_2 x$

$\Rightarrow$ obter $x(t)$

Problema Inverso:

- Dados experimentais
- Parâmetros desconhecidos
- Estimar $\leftarrow$ Ajuste
- Mal-posto (ill-posed)

Exemplo

Dados $x(t)$ experimentais

Modelo: $\frac{dx}{dt} = k_1 - k_2 x$

$\Rightarrow$ encontrar k_1 , k_2

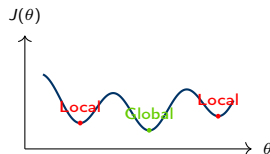
Importante!

O problema inverso é geralmente mais difícil que o direto!

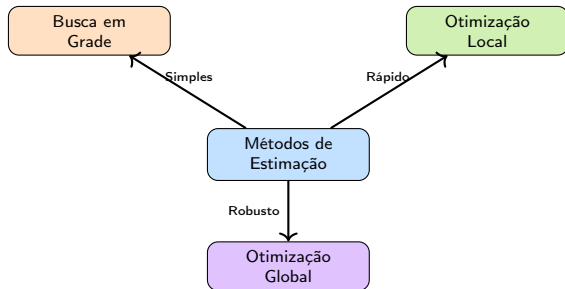
Desafios na Estimação de Parâmetros

Principais Obstáculos

1. **Identificabilidade:** múltiplas soluções
2. **Ruído:** medições imprecisas
3. **Dados escassos:** poucas observações
4. **Mínimos locais:** otimização não-convexa
5. **Correlação:** parâmetros colineares
6. **Custo computacional:** simulações
7. **Incerteza:** quantificação de confiança



Visão Geral dos Métodos



Escolha do Método

- **Poucos parâmetros + boa estimativa:** locais (rápido)
- **Muitos parâmetros + incerteza:** globais (robusto)
- **Exploração inicial:** busca em grade (simples)

Função Objetivo: Conceito

Definição: Função Objetivo (Cost Function)

Medida quantitativa da discrepância entre dados experimentais e previsões do modelo. O objetivo na estimação é encontrar parâmetros que minimizam esta função.

Componentes Principais

- y_i : dados experimentais observados no ponto i
- $\hat{y}_i(\theta)$: previsão ou simulação gerada pelo modelo
- θ : vetor de parâmetros a serem ajustados/estimados
- n : número total de observações experimentais

Função Objetivo: Formulação

A função objetivo quantifica o erro total entre os dados e as previsões:

$$J(\boldsymbol{\theta}) = \text{medida de discrepância entre } \{y_i\} \text{ e } \{\hat{y}_i(\boldsymbol{\theta})\}$$

Problema de Otimização: A estimação é formulada matematicamente como a busca pelo argumento minimizador de $J(\boldsymbol{\theta})$:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Mínimos Quadrados Ordinários (OLS)

Método Mais Comum

Minimização da soma dos quadrados dos desvios entre o modelo e os dados:

$$J(\boldsymbol{\theta}) = \text{SSE}(\boldsymbol{\theta}) = \sum_{i=1}^n [y_i - \hat{y}_i(\boldsymbol{\theta})]^2$$

Resíduo

O resíduo r_i representa o erro de previsão individual no ponto i :

$$r_i = y_i - \hat{y}_i(\boldsymbol{\theta}) \quad \Rightarrow \quad \text{SSE}(\boldsymbol{\theta}) = \sum_{i=1}^n r_i^2$$

Mínimos Quadrados Ordinários (OLS): Propriedades

A formulação OLS clássica possui características importantes:

Vantagens:

- Simples e intuitivo
- Bem estabelecido na literatura
- Solução analítica para modelos lineares
- Interpretação estatística direta

Suposições Estatísticas:

- Erros gaussianos (ruído branco)
- Variância constante (homocedasticidade)
- Erros independentes
- Outliers são muito problemáticos

Mínimos Quadrados Ponderados (WLS)

Quando usar?

Quando as observações têm diferentes níveis de incerteza ou confiabilidade.

$$J(\theta) = \sum_{i=1}^n w_i [y_i - \hat{y}_i(\theta)]^2$$

Pesos w_i :

- $w_i = 1/\sigma_i^2$: inverso da variância (comum)
- $w_i = 1/y_i$: proporcional ao valor (dados log-scale)
- w_i customizado: baseado no erro experimental

Exemplo: Cinética Enzimática

Concentrações baixas têm maior erro relativo.

$\Rightarrow$ usar pesos $w_i = 1/\hat{y}_i$ ou $w_i = 1/\hat{y}_i^2$.

Maximum Likelihood Estimation (MLE): Conceito

Abordagem Estatística

Encontrar parâmetros que maximizam a probabilidade (likelihood) de observar os dados reais obtidos.

Função de Verossimilhança:

$$\mathcal{L}(\boldsymbol{\theta}) = P(\text{dados}|\boldsymbol{\theta})$$

Log-verossimilhança (computacionalmente preferível):

$$\ln \mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n \ln P(y_i|\boldsymbol{\theta})$$

Maximum Likelihood Estimation (MLE): Caso Gaussiano

Caso Gaussiano

Assumindo que os erros de medição ϵ_i são independentes e normalmente distribuídos, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$:

$$\ln \mathcal{L}(\boldsymbol{\theta}) = -\frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n [y_i - \hat{y}_i(\boldsymbol{\theta})]^2$$

Importante!

Sob a hipótese de ruído gaussiano, maximizar a log-verossimilhança $\ln \mathcal{L}(\boldsymbol{\theta})$ é matematicamente equivalente a minimizar a soma dos quadrados dos resíduos (SSE) do método de OLS!

Abordagem Bayesiana: Teorema e Termos

Incorpora Conhecimento Prévio

Usa o Teorema de Bayes para atualizar a nossa crença sobre as distribuições dos parâmetros à medida que novos dados são observados.

$$P(\theta|\text{dados}) = \frac{P(\text{dados}|\theta) \cdot P(\theta)}{P(\text{dados})}$$

- $P(\theta|\text{dados})$: **posterior** (distribuição pós-dados)
- $P(\text{dados}|\theta)$: **likelihood** (verossimilhança dos dados)
- $P(\theta)$: **prior** (conhecimento prévio sobre os parâmetros)
- $P(\text{dados})$: evidência (constante de normalização)

Abordagem Bayesiana: Vantagens e Desafios

Vantagens

- Permite incorporar dados históricos ou da literatura nas distribuições priors
- Quantifica incerteza de forma natural (probabilidade dos parâmetros)
- Previne overfitting agindo como uma regularização natural

Desafio

Requer a escolha de distribuições priors adequadas e demanda alto custo computacional com algoritmos de amostragem (como MCMC).

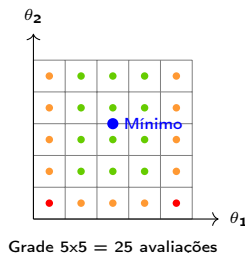
Conceito de Busca em Grade

Definição: Grid Search

Método de otimização que avalia sistematicamente a função objetivo em uma grade regular de valores de parâmetros.

Algoritmo:

1. Definir limites para os parâmetros
2. Criar grade regular com $n_1 \times n_2 \times \dots$ pontos
3. Avaliar a função objetivo $J(\theta)$ em cada nó da grade
4. Selecionar o ponto com o menor valor de J



Vantagens e Desvantagens

Vantagens

- Simples de implementar
- Facilmente paralelizável
- Garantia de encontrar o mínimo global
- Sem derivadas necessárias

Desvantagens

- Custo exponencial: $O(n^p)$
- Inviável para muitos parâmetros
- Resolução da grade limitada
- Maldição da dimensionalidade

Grid Search: Custo Computacional

O custo computacional cresce exponencialmente com o número de parâmetros estimáveis.

Importante!

Custo Computacional: assumindo 10 valores testados por parâmetro:

- 2 parâmetros: $10^2 = 100$ avaliações de J
- 5 parâmetros: $10^5 = 100.000$ avaliações de J
- 10 parâmetros: 10^{10} avaliações (impraticável em tempo viável!)

Conclusão: A busca em grade é útil para exploração inicial de poucos parâmetros (1 ou 2), mas inadequada para problemas de alta dimensão.

Implementação: Grid Search 1D

```
1 import numpy as np
2
3 def objective_function(theta):
4     return (theta - 3)**2 + 5
5
6 # Definir grade e avaliar
7 theta_values = np.linspace(0, 6, 50)
8 J_values = [objective_function(t) for t in theta_values]
9
10 # Encontrar minimo
11 idx_min = np.argmin(J_values)
12 theta_best = theta_values[idx_min]
13 J_best = J_values[idx_min]
14
15 print(f"Parametro otimo: {theta_best:.3f}")
16 print(f"Valor da funcao: {J_best:.3f}")
```

Listing 1: Busca em grade unidimensional

Implementação: Grid Search 2D

```
1 def sse_model(params, t, y):
2     a, b = params
3     return np.sum((y - a * np.exp(-b * t))**2)
4
5 # Dados experimentais e grade
6 t = np.array([0, 1, 2, 3, 4, 5])
7 y = np.array([10, 6.1, 3.7, 2.2, 1.4, 0.8])
8 a_range = np.linspace(5, 15, 20)
9 b_range = np.linspace(0.1, 1.0, 20)
10 SSE_grid = np.array([[sse_model([a, b], t, y) for b in b_range] for a in a_range
11                        ])
12
13 # Encontrar minimo
14 idx = np.unravel_index(SSE_grid.argmin(), SSE_grid.shape)
15 a_best, b_best = a_range[idx[0]], b_range[idx[1]]
16 print(f"Parametros otimos: a={a_best:.3f}, b={b_best:.3f}")
```

Listing 2: Busca em grade bidimensional

Visualização: Contour Plot

```
1 # Criar contour plot
2 A, B = np.meshgrid(a_range, b_range)
3
4 plt.figure(figsize=(8, 5))
5 contour = plt.contourf(A, B, SSE_grid.T, levels=20, cmap='viridis')
6 plt.colorbar(contour, label='SSE')
7
8 # Marcar minimo e contornos
9 plt.plot(a_best, b_best, 'r*', markersize=15,
10          label=f'Minimo: ({a_best:.2f}, {b_best:.2f})')
11 plt.contour(A, B, SSE_grid.T, levels=10,
12             colors='white', linewidths=0.5, alpha=0.5)
13
14 plt.xlabel('Parametro a')
15 plt.ylabel('Parametro b')
16 plt.title('Landscape da Funcao Objetivo')
17 plt.legend()
18 plt.grid(True, alpha=0.3)
```

Listing 3: Mapa de contorno da funcao objetivo

Gradient Descent (Descida do Gradiente)

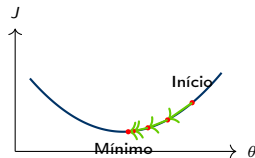
Definição: Gradient Descent

Método iterativo que se move na direção oposta ao gradiente para encontrar um mínimo local da função objetivo.

$$\theta^{(k+1)} = \theta^{(k)} - \alpha \nabla J(\theta^{(k)})$$

Componentes:

- $\theta^{(k)}$: parâmetros na iteração k
- $\alpha > 0$: taxa de aprendizado (passo)
- $\nabla J = \left[\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots \right]^T$: vetor gradiente



Escolha da Taxa de Aprendizado: Impacto



α pequeno
Lento



α ideal
Rápido



α grande
Oscila

Escolha da Taxa de Aprendizado: Estratégias

Importante!

A escolha de α é crucial para convergência!

Estratégias

- **Fixo:** α constante (simples, pode não convergir)
- **Decrescente:** $\alpha_k = \alpha_0 / (1 + k)$ (garante convergência)
- **Line search:** otimizar α em cada iteração (mais robusto)
- **Adaptativo:** ajustar baseado em progresso (Adam, RMSprop)

Critérios de Convergência

Quando Parar?

Teste múltiplos critérios simultaneamente:

1. **Gradiente pequeno:** $\|\nabla J(\boldsymbol{\theta}^{(k)})\| < \epsilon_g$
 - Indica ponto estacionário
2. **Mudança em parâmetros:** $\|\boldsymbol{\theta}^{(k+1)} - \boldsymbol{\theta}^{(k)}\| < \epsilon_\theta$
 - Parâmetros não mudam mais
3. **Mudança na função:** $|J^{(k+1)} - J^{(k)}| < \epsilon_J$
 - Função objetivo estabilizou
4. **Número máximo de iterações:** $k > k_{\max}$
 - Prevenir loops infinitos

Valores Típicos

$$\epsilon_g = 10^{-6}, \epsilon_\theta = 10^{-6}, \epsilon_J = 10^{-8}, k_{\max} = 1000$$

Implementação: Gradient Descent

```
1 def gradient_descent(f, theta0, alpha=0.01, max_iter=1000, tol=1e-6):
2     theta = np.array(theta0, dtype=float)
3     history = [theta.copy()]
4
5     for k in range(max_iter):
6         # Calcular gradiente numericamente
7         grad = numerical_gradient(f, theta)
8
9         # Atualizar parametros e checar convergencia
10        theta_new = theta - alpha * grad
11        if np.linalg.norm(theta_new - theta) < tol:
12            print(f"Convergiu em {k} iteracoes")
13            break
14
15        theta = theta_new
16        history.append(theta.copy())
17
18    return theta, np.array(history)
```

Listing 4: Descida do gradiente principal

Implementação: Gradiente Numérico

```
1 def numerical_gradient(f, theta, h=1e-5):
2     """Gradiente por diferencas finitas"""
3     grad = np.zeros_like(theta)
4     for i in range(len(theta)):
5         theta_plus = theta.copy()
6         theta_plus[i] += h
7         grad[i] = (f(theta_plus) - f(theta)) / h
8     return grad
```

Listing 5: Calculo numerico do gradiente

Método de Newton-Raphson

Ideia

Usar informação de segunda ordem (curvatura) para convergência mais rápida

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \mathbf{H}^{-1}(\boldsymbol{\theta}^{(k)}) \nabla J(\boldsymbol{\theta}^{(k)})$$

- H: matriz Hessiana (derivadas segundas)
- $H_{ij} = \frac{\partial^2 J}{\partial \theta_i \partial \theta_j}$

Vantagens:

- Convergência quadrática
- Rápido perto do mínimo
- Não precisa de step size

Desvantagens:

- Calcular H é caro
- Inverter H também
- Não-convergência longe do mínimo

Método de Gauss-Newton

Específico para Mínimos Quadrados

Aproxima a Hessiana usando apenas o Jacobiano das previsões do modelo.

Para $J(\boldsymbol{\theta}) = \sum r_i^2(\boldsymbol{\theta})$ com resíduos $r_i = y_i - \hat{y}_i(\boldsymbol{\theta})$:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + (\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \mathbf{r}$$

Onde:

- $\mathbf{J}$: Jacobiana ($J_{ij} = \partial r_i / \partial \theta_j$)
- $\mathbf{r}$: vetor de resíduos individuais

Vantagens

- Evita o cálculo da Hessiana completa
- Excelente para mínimos quadrados
- Base do algoritmo Levenberg-Marquardt

Levenberg-Marquardt (LM): Algoritmo

Definição: Levenberg-Marquardt

Combina Gauss-Newton e gradient descent, interpolando entre ambos via parâmetro de amortecimento.

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + (\mathbf{J}^T \mathbf{J} + \lambda \mathbf{I})^{-1} \mathbf{J}^T \mathbf{r}$$

- λ : parâmetro de amortecimento (damping)
- λ grande $\Rightarrow$ gradient descent (pequenos passos, robusto)
- λ pequeno $\Rightarrow$ Gauss-Newton (passos grandes, rápido)

Levenberg-Marquardt (LM): Estratégia

Estratégia Adaptativa de Ajuste:

- Se J diminui: aceitar passo, diminuir λ (mais agressivo)
- Se J aumenta: rejeitar passo, aumentar λ (mais conservador)

Importante!

LM é o método padrão para ajuste não-linear de curvas!

Otimização Local: Michaelis-Menten (curve_fit)

```
1 from scipy.optimize import curve_fit
2 import numpy as np
3
4 # Modelo de Michaelis-Menten
5 def michaelis_menten(S, Vmax, Km):
6     return Vmax * S / (Km + S)
7
8 # Dados experimentais
9 S_data = np.array([0.5, 1, 2, 5, 10, 20, 50])
10 v_data = np.array([0.4, 0.7, 1.2, 1.8, 2.1, 2.3, 2.4])
11
12 # Otimizacao via curve_fit
13 params, cov = curve_fit(michaelis_menten, S_data, v_data,
14                          p0=[3, 5])
15 Vmax_fit, Km_fit = params
16
17 print(f"Vmax = {Vmax_fit:.3f}")
18 print(f"Km = {Km_fit:.3f}")
```

Listing 6: Ajuste simples usando curve_fit

Otimização Local: Michaelis-Menten (least_squares)

```
1 from scipy.optimize import least_squares
2 import numpy as np
3
4 # Reuso dos dados anteriores e definicao de residuos
5 def residuals(params):
6     Vmax, Km = params
7     return v_data - michaelis_menten(S_data, Vmax, Km)
8
9 result = least_squares(residuals, x0=[3, 5], method='lm')
10 print(f"Parametros otimos: {result.x}")
11 print(f"Custo final: {result.cost:.6f}")
12 print(f"Sucesso: {result.success}")
```

Listing 7: Ajuste via least_squares (LM)

Visualização do Ajuste

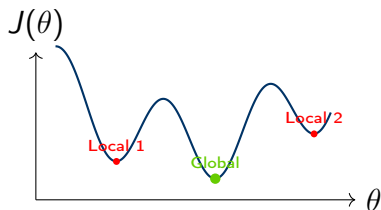
```
1 # Gerar curva do modelo ajustado
2 S_smooth = np.linspace(0, 60, 200)
3 v_fit = michaelis_menten(S_smooth, Vmax_fit, Km_fit)
4
5 # Plotar dados e ajuste
6 plt.figure(figsize=(10, 6))
7 plt.plot(S_data, v_data, 'ko', markersize=10, label='Dados exp')
8 plt.plot(S_smooth, v_fit, 'b-', linewidth=2,
9          label=f'Ajuste: Vmax={Vmax_fit:.2f}, Km={Km_fit:.2f}')
10
11 # Linhas de referencia
12 plt.axhline(y=Vmax_fit, color='r', linestyle='--', alpha=0.5, label='Vmax')
13 plt.axvline(x=Km_fit, color='g', linestyle='--', alpha=0.5, label='Km')
14 plt.axhline(y=Vmax_fit/2, color='gray', linestyle=':')
15
16 plt.xlabel('[Substrato] (mM)'); plt.ylabel('Velocidade (umol/min)')
17 plt.title('Ajuste de Michaelis-Menten')
18 plt.legend(); plt.grid(True, alpha=0.3)
```

Listing 8: Comparação dados vs modelo ajustado

Por que Otimização Global?

Problema

Métodos locais podem ficar presos em mínimos locais



Quando Usar?

- Função objetivo altamente não-linear
- Múltiplos mínimos locais
- Sem boa estimativa inicial
- Parâmetros com grande incerteza
- Modelos biológicos complexos

Algoritmos Genéticos (GA)

Definição: Algoritmo Genético

Método inspirado na evolução biológica que mantém uma população de soluções candidatas e as evolui usando seleção, cruzamento e mutação.

Analogia Biológica:

- **Indivíduo:** parâmetros θ
- **Gene:** parâmetro individual θ_i
- **Fitness:** inverso da função objetivo $1/J(\theta)$
- **População:** conjunto de soluções

Operadores Genéticos

1. **Seleção:** melhores soluções
2. **Cruzamento:** combinar dois pais
3. **Mutação:** variação aleatória
4. **Elitismo:** manter o campeão

Simulated Annealing (SA): Conceito

Inspiração

Processo de recozimento em metalurgia: aquecer metal e esfriar lentamente para alcançar estrutura cristalina de mínima energia.

Ideia Principal:

- Permitir movimentos para soluções piores com probabilidade decrescente.
- A "Temperatura" (T) controla a aceitação de soluções piores.
- Alta temperatura: exploração ampla do espaço.
- Baixa temperatura: refinamento e convergência local.

Simulated Annealing (SA): Algoritmo

Probabilidade de Aceitação:

$$P(\text{aceitar}) = \begin{cases} 1 & \text{se } \Delta J < 0 \\ e^{-\Delta J/T} & \text{se } \Delta J \geq 0 \end{cases}$$

onde $\Delta J = J(\theta_{\text{novo}}) - J(\theta_{\text{atual}})$ e T é a temperatura.

Cronograma de Resfriamento

$$T_k = T_0 \cdot \alpha^k \text{ onde } 0 < \alpha < 1 \text{ (tipicamente } \alpha \in [0.8, 0.99])$$

Differential Evolution (DE)

Definição: Evolução Diferencial

Algoritmo evolutivo que cria novos candidatos através de diferenças entre membros da população.

Operação Principal:

1. Escolher 3 indivíduos: $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$
2. Mutação: $\mathbf{v} = \mathbf{a} + F(\mathbf{b} - \mathbf{c})$
3. Cruzamento usando taxa CR
4. Seleção: manter o melhor
 - Fator de escala $F \in [0.5, 1.0]$
 - Crossover $CR \in [0.5, 0.9]$

Vantagens

- Simples de implementar
- Poucos parâmetros de ajuste
- Excelente robustez
- Não requer gradientes
- Ótimo para múltiplos mínimos

Particle Swarm Optimization (PSO)

Inspiração

Comportamento social de bandos de pássaros ou cardumes de peixes.

Conceito:

- Cada partícula tem posição $\mathbf{x}_i$ e velocidade $\mathbf{v}_i$.
- Movimento influenciado por: melhor posição própria ($\mathbf{p}_i$) e melhor posição global ($\mathbf{g}$).

$$\begin{aligned}\mathbf{v}_i^{(k+1)} &= w\mathbf{v}_i^{(k)} + c_1r_1(\mathbf{p}_i - \mathbf{x}_i^{(k)}) + c_2r_2(\mathbf{g} - \mathbf{x}_i^{(k)}) \\ \mathbf{x}_i^{(k+1)} &= \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+1)}\end{aligned}$$

- w : peso de inércia | c_1, c_2 : coeficientes cognitivo e social
- r_1, r_2 : números aleatórios uniformes em $[0, 1]$

Implementação: Differential Evolution (Função)

```
1 from scipy.optimize import differential_evolution
2 import numpy as np
3
4 # Funcao objetivo
5 def objective(params, t_data, y_data):
6     """Modelo exponencial:  $y = a * \exp(-b*t)$ """
7     a, b = params
8     y_pred = a * np.exp(-b * t_data)
9     sse = np.sum((y_data - y_pred)**2)
10    return sse
11
12 # Dados experimentais
13 t_data = np.array([0, 1, 2, 3, 4, 5])
14 y_data = np.array([10, 6.1, 3.7, 2.2, 1.4, 0.8])
```

Listing 9: Modelo e funcao objetivo para DE

Implementação: Differential Evolution (Otimização)

```
1 # Definir bounds para parametros: [(a_min, a_max), (b_min, b_max)]
2 bounds = [(1, 20), (0.01, 2.0)]
3
4 # Otimizar com DE
5 result = differential_evolution(
6     objective, bounds, args=(t_data, y_data),
7     strategy='best1bin', maxiter=1000, popsize=15,
8     tol=1e-7, mutation=(0.5, 1), recombination=0.7, seed=42
9 )
10
11 print(f"Parametros otimos: a={result.x[0]:.3f}, b={result.x[1]:.3f}")
12 print(f"SSE minimo: {result.fun:.6f}")
13 print(f"Convergiu: {result.success}")
```

Listing 10: Otimizacao global usando scipy

Comparação de Métodos de Otimização

Método	Velocidade	Global	Derivadas	Parâmetros	Uso
Grid Search	—	Sim	Não	Poucos	Exploração
Gradient Descent	++	Não	Sim	Média	Local
Newton-Raphson	+++	Não	2ª ordem	Média	Refinamento
Levenberg-Marquardt	+++	Não	Jacobiano	Média	Mín. Quad.
GA	+	Sim	Não	Muitos	Robusto
Simulated Annealing	+	Sim	Não	Muitos	Explora
Differential Evolution	++	Sim	Não	Média	Melhor
Particle Swarm	++	Sim	Não	Média	Rápido

Recomendações Práticas

- **Boa estimativa inicial + poucos parâmetros:** Use Levenberg-Marquardt (rápido e local).
- **Sem estimativa inicial + mínimos locais:** Use Differential Evolution (robusto e global).
- **Problema altamente não-linear:** Abordagem híbrida (rodar DE primeiro e usar resultado como inicial no LM).
- **Muitos parâmetros (> 20):** Considere métodos Bayesianos (MCMC).

Identificabilidade Estrutural: Exemplos

Definição: Identificabilidade Estrutural

Propriedade teórica do modelo: os parâmetros podem ser determinados unicamente a partir de dados perfeitos (infinitos e sem ruído)?

Identificável

$$\frac{dx}{dt} = k_1 - k_2x$$

Ambos k_1 e k_2 podem ser determinados unicamente.

Não Identificável

$$\frac{dx}{dt} = (k_1k_2) - k_2x$$

Apenas o produto k_1k_2 é identificável, não k_1 e k_2 separadamente.

Importante!

Antes de estimar parâmetros, verifique se o modelo é estruturalmente identificável!

Ferramentas de Análise:

- **Análise algébrica:** eliminação de variáveis via álgebra computacional.
- **Softwares Especializados:** DAISY, GenSSI, STRIKE-GOLDD.

Definição: Identificabilidade Prática

Podem os parâmetros ser determinados com precisão razoável a partir de dados experimentais reais (finitos e com ruído)?

Causas de Não-Identificabilidade:

- Dados insuficientes/baixa qualidade
- Parâmetros altamente correlacionados
- Modelo super-parametrizado (overfitting)
- Baixa sensibilidade a parâmetros

Exemplo: Correlação de Parâmetros

Modelo: $y = a \cdot e^{-b \cdot t}$

Se t é pequeno: $e^{-bt} \approx 1 - bt$

$\Rightarrow y \approx a(1 - bt) = a - abt$

Difícil separar a de ab (produto correlacionado).

Matriz de Informação de Fisher (FIM)

Quantifica Informação sobre Parâmetros

Mede quanta informação os dados contêm sobre cada parâmetro.

$$F_{ij} = \sum_{k=1}^n \frac{1}{\sigma_k^2} \frac{\partial \hat{y}_k}{\partial \theta_i} \frac{\partial \hat{y}_k}{\partial \theta_j}$$

Onde:

- $\hat{y}_k$: previsão do modelo no ponto k
- σ_k^2 : variância do erro no ponto k
- θ_i, θ_j : parâmetros

Interpretação:

- FIM grande $\Rightarrow$ alta precisão
- FIM pequena $\Rightarrow$ alta incerteza
- $\det(F) \approx 0 \Rightarrow$ não-identificável

Intervalos de Confiança: Teoria

Estimativa Assintótica

Para grandes amostras, $\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\theta}^*, \mathbf{C})$ onde:

$$\mathbf{C} = \text{Cov}(\boldsymbol{\theta}) \approx \mathbf{F}^{-1}$$

Intervalo de Confiança (95%):

$$\theta_i^* \pm 1.96 \sqrt{C_{ii}}$$

Intervalos de Confiança: Aplicação Prática

Na Prática

- Diagonal: $\sqrt{C_{ii}}$ = erro padrão do parâmetro i
- Off-diagonal: C_{ij} = correlação entre parâmetros i e j

Importante!

Intervalos de confiança estreitos $\nRightarrow$ modelo correto!
Apenas indicam precisão do ajuste.

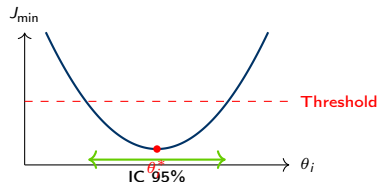
Profile Likelihood

Definição: Profile Likelihood

Método robusto para calcular intervalos de confiança explorando o espaço de parâmetros sistematicamente.

Procedimento para θ_i :

1. Fixar θ_i em valor específico
2. Otimizar os outros parâmetros
3. Registrar $J_{\min}(\theta_i)$
4. Repetir para vários valores
5. Plotar $J_{\min}$ vs θ_i



Implementação: Visualização de Correlação

```
1 from scipy.optimize import curve_fit
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # Ajustar e extrair correlacao
6 params, cov = curve_fit(model_func, x_data, y_data)
7 std = np.sqrt(np.diag(cov))
8 corr = cov / np.outer(std, std)
9
10 # Plotar heatmap
11 param_names = ['k1', 'k2', 'k3']
12 plt.figure(figsize=(8, 6))
13 sns.heatmap(corr, annot=True, fmt='.2f',
14             xticklabels=param_names, yticklabels=param_names,
15             cmap='coolwarm', center=0, vmin=-1, vmax=1)
16 plt.title('Matriz de Correlacao')
```

Listing 11: Heatmap de correlacao usando seaborn

Implementação: Alertas de Correlação

```
1 # Alerta de alta correlacao
2 for i in range(len(params)):
3     for j in range(i+1, len(params)):
4         if abs(corr[i,j]) > 0.9:
5             print(f"ALERTA: {param_names[i]} e {param_names[j]} "
6                   f"altamente correlacionados (r={corr[i,j]:.2f})")
```

Listing 12: Detecção de parâmetros colineares

Implementação: Cálculo de Intervalos de Confiança

```
1 from scipy import stats
2 import numpy as np
3
4 params_best = params
5 errors = np.sqrt(np.diag(cov))
6
7 # Intervalo de confiança 95%
8 alpha = 0.05
9 t_val = stats.t.ppf(1-alpha/2, len(x_data)-len(params))
10 ci = t_val * errors
```

Listing 13: Calcular ICs usando distribuicao t

Implementação: Plot de Parâmetros e CIs

```
1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots(figsize=(10, 6))
4 x_pos = np.arange(len(param_names))
5
6 ax.bar(x_pos, params_best, yerr=ci, alpha=0.7, capsize=10,
7       color='steelblue', error_kw={'elinewidth': 2, 'capthick': 2})
8 ax.set_xticks(x_pos)
9 ax.set_xticklabels(param_names)
10 ax.set_ylabel('Valor do Parametro')
11 ax.set_title('Parametros Estimados com ICs 95%')
12 ax.grid(axis='y', alpha=0.3)
13
14 for name, val, err in zip(param_names, params_best, ci):
15     print(f"{name} = {val:.4f} +/- {err:.4f} "
16           f"({100*err/val:.1f}% erro relativo)")
```

Listing 14: Visualizar parametros com barras de erro

Verificação de Hipóteses

Examinar resíduos $r_i = y_i - \hat{y}_i$ para validar suposições do modelo.

Padrões a Verificar:

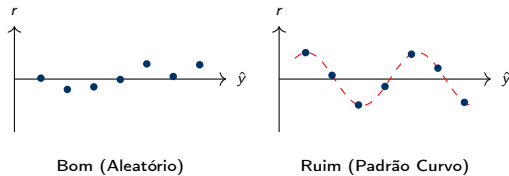
Bom Ajuste

- Distribuição aleatória
- Sem padrão sistemático
- Centrados em zero
- Variância constante
- Aproximadamente normais

Problemas

- Padrão curvo: inadequação
- Funil: heterocedasticidade
- Outliers: dados anômalos
- Não-normalidade do erro

Análise de Resíduos: Visualização



Coeficiente de Determinação (R^2): Definição

Definição: R^2

Proporção da variância nos dados explicada pelo modelo. Varia de 0 (nenhum ajuste) a 1 (ajuste perfeito).

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- SS_{res} : soma dos quadrados dos resíduos
- SS_{tot} : soma dos quadrados totais
- $\bar{y}$: média dos dados experimentais

Coeficiente de Determinação (R^2): Limitações

Limitações Importantes

- R^2 sempre aumenta ao adicionar novos parâmetros (mesmo irrelevantes).
- Um R^2 alto não garante que o modelo é correto ou preditivo.
- Pode ser altamente enganoso para modelos não-lineares.
- Use o R^2 ajustado para comparar modelos com diferentes números de parâmetros.

R^2 Ajustado: Definição

Penaliza Complexidade do Modelo

Ajusta o R^2 baseando-se no número de graus de liberdade.

$$R_{\text{adj}}^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

onde:

- n : número de observações (tamanho da amostra)
- p : número de parâmetros estimáveis

Características:

- Pode diminuir ao adicionar parâmetros ruins ou irrelevantes.
- Mais apropriado para comparação de modelos.

Akaike Information Criterion (AIC): Teoria

Definição: AIC

Critério que balanceia a qualidade do ajuste com a complexidade do modelo.

$$AIC = 2p - 2 \ln(\mathcal{L}) \approx n \ln \left(\frac{SSE}{n} \right) + 2p$$

Onde:

- p : número de parâmetros
- $\mathcal{L}$: máxima verossimilhança
- SSE: soma dos resíduos quadrados

Interpretação:

- Menor AIC = melhor modelo
- Penaliza complexidade extra
- $\Delta AIC > 10$: evidência forte

AIC Corrigido (AICc)

Para amostras pequenas ($n/p < 40$):

$$AICc = AIC + \frac{2p(p+1)}{n-p-1}$$

Bayesian Information Criterion (BIC): Definição

Definição: BIC (ou Schwarz Criterion)

Similar ao AIC, mas penaliza mais fortemente a complexidade do modelo.

$$\text{BIC} = p \ln(n) - 2 \ln(\mathcal{L}) \approx n \ln \left(\frac{\text{SSE}}{n} \right) + p \ln(n)$$

Comparação AIC vs BIC:

- BIC penaliza complexidade mais forte (termo $p \ln(n)$ vs $2p$ do AIC).
- Para $n > 7$, o BIC seleciona modelos mais parcimoniosos (menores).
- O AIC busca otimizar a previsão futura, enquanto o BIC tenta identificar o "modelo verdadeiro".

Comparação de Critérios de Seleção (AIC vs BIC)

Critério	Objetivo	Preferência
AIC	Previsão / Incerteza	Modelos maiores
BIC	Identificação do Processo	Modelos menores

Resumo Prático

Na biologia de sistemas, onde os dados costumam ser limitados e o ruído é alto, o **AICc** e o **BIC** são usados em conjunto. Se divergirem, prioriza-se o modelo mais simples (BIC) para evitar overfitting.

Cross-Validation (Validação Cruzada)

Teste de Capacidade Preditiva

Avaliar o desempenho do modelo em dados não vistos durante o treinamento para evitar overfitting.

K-Fold Cross-Validation:

1. Dividir dados em K folds.
2. Para cada fold k :
 - Treinar modelo nos outros $K - 1$ folds.
 - Testar no fold k .
 - Registrar erro de previsão.
3. Calcular erro médio de CV.

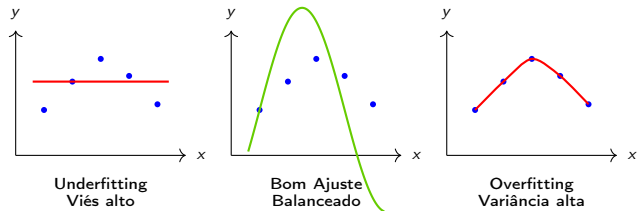
Train Train Test Train Train

5-Fold CV (Dobra 3 como teste)

Diretrizes

Típico: $K = 5$ ou $K = 10$. Se $K = n$, chamamos de Leave-One-Out (LOO), muito custoso.

Overfitting vs Underfitting



Sintomas de Overfitting

- Erro de treinamento muito menor que o erro de teste.
- Modelo complexo com excesso de parâmetros livres.
- ICs excessivamente largos; previsões ruins fora do domínio.

Implementação: Cálculo de Métricas (RMSE, R2)

```
1 import numpy as np
2
3 def calculate_metrics(y_true, y_pred, n_params):
4     n = len(y_true)
5     residuals = y_true - y_pred
6
7     # SSE e MSE
8     sse = np.sum(residuals**2)
9     mse = sse / n
10    rmse = np.sqrt(mse)
11
12    # R-squared e R-squared ajustado
13    ss_tot = np.sum((y_true - np.mean(y_true))**2)
14    r2 = 1 - (sse / ss_tot)
15    r2_adj = 1 - (1-r2)*(n-1)/(n-n_params-1)
```

Listing 15: Cálculo de RMSE e R-squared ajustado

Implementação: Cálculo de Métricas (AIC, BIC)

```
1  # Calculo continuacao: AIC e BIC
2  aic = n*np.log(sse/n) + 2*n_params
3  bic = n*np.log(sse/n) + n_params*np.log(n)
4
5  return {
6      'SSE': sse, 'MSE': mse, 'RMSE': rmse,
7      'R2': r2, 'R2_adj': r2_adj, 'AIC': aic, 'BIC': bic,
8      'residuals': residuals
9  }
10
11 # Executar e exibir resultados
12 metrics = calculate_metrics(y_data, y_pred, n_params=2)
13 for key, val in metrics.items():
14     if key != 'residuals':
15         print(f"{key}: {val:.4f}")
```

Listing 16: Cálculo de AIC/BIC e execucao

Exercícios - Parte 1: Busca em Grade

Questão 1: Busca em Grade

Implemente busca em grade para encontrar parâmetros a e b do modelo $y = a \cdot x^b$ usando os dados:

x	1	2	3	4	5	6
y	2.1	8.3	18.2	32.5	50.1	72.3

Use: $a \in [1, 3]$, $b \in [1.5, 2.5]$, grade 20x20.

Questão 2: Gradient Descent

Implemente gradient descent manualmente para minimizar $J(\theta) = (\theta - 5)^2 + 3$:

1. Use $\theta_0 = 0$ e $\alpha = 0.1$
2. Plote trajetória de convergência
3. Teste diferentes valores de α (0.01, 0.1, 0.5, 1.0)
4. Compare número de iterações até convergência

Questão 3: Levenberg-Marquardt

Ajuste parâmetros da cinética de Michaelis-Menten $v = \frac{V_{\max}[S]}{K_M + [S]}$ aos dados:

[S] (mM)	0.1	0.5	1	2	5	10	20
v ($\mu\text{mol}/\text{min}$)	0.3	1.1	1.8	2.5	3.3	3.7	3.9

1. Use `scipy.optimize.curve_fit`
2. Calcule intervalos de confiança
3. Plote dados vs modelo ajustado e faça análise de resíduos

Questão 4: Differential Evolution

Compare DE com LM no problema anterior:

1. Use bounds amplos: $V_{\max} \in [0.1, 10]$, $K_M \in [0.1, 10]$
2. Qual método é mais rápido? Qual encontra melhor solução?
3. Use DE para encontrar boa estimativa inicial para o LM

Questão 5: Crescimento Logístico

Considere dados de crescimento bacteriano:

1. Gere dados sintéticos: $N(t) = \frac{K}{1 + \left(\frac{K}{N_0} - 1\right)e^{-rt}}$ com $N_0 = 1$, $r = 0.5$, $K = 100$, $t \in [0, 20]$ + ruído.
2. Estime r e K usando três métodos diferentes.
3. Analise identificabilidade e profile likelihood.

Questão 6: Validação Cruzada

Implemente 5-fold cross-validation para o modelo logístico:

1. Divida dados em 5 folds.
2. Calcule RMSE médio de CV.
3. Compare com RMSE de treinamento (checar overfitting).
4. Teste com modelos de complexidade crescente.

Projeto: Sistema de Lotka-Volterra

Estime parâmetros do sistema predador-presa:

$$\begin{aligned}\frac{dV}{dt} &= \alpha V - \beta VP \\ \frac{dP}{dt} &= \delta \beta VP - \gamma P\end{aligned}$$

Dados simulados: Gere séries temporais com parâmetros conhecidos + ruído gaussiano.

Tarefas Recomendadas

1. Implementar função de custo (SSE de ambas variáveis).
2. Estimar 4 parâmetros usando Differential Evolution.
3. Refinar estimativas com Levenberg-Marquardt.
4. Analisar identificabilidade (matriz de correlação de Fisher).
5. Calcular intervalos de confiança assintóticos e plotá-los.
6. Validar o ajuste simulando o modelo vs dados ruidosos.

Referências Principais

-  Nocedal J., Wright S.J. (2006). *Numerical Optimization*, 2nd ed. Springer.
-  Raue A. et al. (2009). Structural and practical identifiability analysis of partially observed dynamical models. *Bioinformatics*.
-  Voit E.O. et al. (2015). 150 years of the mass action law. *PLoS Computational Biology*.
-  Moles C.G. et al. (2003). Parameter estimation in biochemical pathways. *Genome Research*.
-  Burnham K.P., Anderson D.R. (2002). *Model Selection and Multimodel Inference*, 2nd ed. Springer.

Livros

- Press W.H. et al. (2007). *Numerical Recipes*. Cambridge.
- Bard Y. (1974). *Nonlinear Parameter Estimation*.
- Seber G.A.F., Wild C.J. (2003). *Nonlinear Regression*. Wiley.

Artigos Importantes

- Gutenkunst R.N. (2007). Universally sloppy parameter sensitivities.
- Villaverde A.F. (2016). Cooperative parameter estimation.

Software e Tutoriais

- **SciPy**: <https://docs.scipy.org/doc/scipy/reference/optimize.html>
- **lmfit**: <https://lmfit.github.io>
- **PyMC**: <https://www.pymc.io>

Obrigado!

Capítulo 5: Estimação de Parâmetros

Próximo Capítulo:
Sistemas Gênicos

Dúvidas? Perguntas?